

Stored Procedures no MySQL

Prof.^a M.^a Mariana Meirelles de Mello
Oliveira



Cliente (Aplicação)

Chamada da rotina armazenada com parâmetros.



Stored Procedure

Lógica de negócios centralizada e pré-compilada.



```
CREATE PROCEDURE obter_dados()  
BEGIN  
    SELECT * FROM tabelas;  
END;
```



O que é Stored Procedure?



Armazenamento

Bloco de código SQL que fica fisicamente armazenado e compilado no banco de dados.



Execução

O código é executado estritamente sob demanda demanda pela aplicação ou ou diretamente pelo usuário.



Padronização

Permite ampla reutilização reutilização de rotinas complexas e garante a padronização das regras. regras.

Por que usar?

Quatro pilares estratégicos para otimizar o desenvolvimento do seu projeto



Reutilização de código

Reduz a redundância e acelera o ciclo de desenvolvimento reaproveitando componentes e módulos previamente estabelecidos.



Segurança

Implementa múltiplas camadas de proteção consolidadas para mitigar vulnerabilidades e prevenir acessos não autorizados.



Performance

Otimiza a alocação de recursos e melhora o tempo de resposta, garantindo uma execução rápida e altamente eficiente.



Centralização da lógica

Facilita a manutenção e a escalabilidade ao concentrar todas as regras de negócio complexas em um único núcleo gerenciável.

Sintaxe Básica

1. Alteração de Delimitador

O uso de DELIMITER \$\$ evita que o MySQL encerre a execução prematuramente ao encontrar um ponto e vírgula dentro do procedimento.

2. Escopo de Execução

As palavras reservadas BEGIN e END encapsulam todos os comandos SQL que compõem a lógica interna do procedimento.

3. Restauração do Padrão

Finalizamos com DELIMITER ; para retornar o interpretador ao seu comportamento normal de execução de linha única.

 procedure.sql

```
DELIMITER $$  
  
CREATE PROCEDURE nome()  
  
BEGIN  
    -- comandos SQL  
  
END $$  
  
DELIMITER ;
```

Exemplo Simples

listar_moradores.sql

```
CREATE PROCEDURE listar_moradores()
```

```
BEGIN
```

```
    SELECT * FROM morador;
```

```
END
```

Declaração da Procedure

O comando **CREATE PROCEDURE** inicia a definição, definição, atribuindo o nome **listar_moradores** à rotina armazenada.

Bloco de Execução

As palavras-chave **BEGIN** e **END** são utilizadas para delimitar o corpo do procedimento onde as instruções residem.

Consulta Principal

O núcleo é a instrução **SELECT * FROM morador, morador**, responsável por retornar todos os registros contidos na tabela especificada.

Classificação de Parâmetros

Tipos de fluxo de dados em procedimentos e funções de banco de dados



IN

Entrada

O valor é passado para o procedimento, mas não pode ser modificado internamente.



OUT

Saída

O valor é retornado pelo procedimento para o programa ou ambiente chamador.



INOUT

Entrada e Saída

Fornece um valor inicial ao procedimento e pode retornar um valor modificado.

Parâmetro IN na Prática

```
CREATE PROCEDURE buscar_morador (IN
nome VARCHAR(100))

SELECT * FROM morador WHERE nome =
nome;
```



O Parâmetro "IN"

O modificador IN indica que o parâmetro serve apenas para receber um valor de quem chama a procedure, atuando como um dado de entrada estrito.



Definição de Tipo

É obrigatório definir o tipo de dado do parâmetro de entrada, neste caso, VARCHAR(100), garantindo a integridade dos dados durante a execução.



Filtro Condicional

O valor recebido no parâmetro "nome" é imediatamente utilizado na cláusula WHERE para filtrar os registros da tabela "morador".

Exemplo OUT



Parâmetro OUT

Define a variável que será responsável por exportar o valor calculado para fora da rotina.



Cláusula INTO

Direciona o resultado da contagem diretamente para a variável declarada.



```
CREATE PROCEDURE contar_moradores (OUT  
total INT)  
  
SELECT COUNT(*) INTO total  
  
FROM morador;
```


Exemplo INOUT



Parâmetro INOUT

É informado um valor que é modificado dentro da procedure e esta retorna um novo valor



Exemplo

Em um sistema de gestão de eventos, a organização pode decidir aumentar a quantidade de vagas disponíveis para um evento específico.



```
CREATE PROCEDURE atualizar_vagas (IN  
p_idEvento INT, INOUT p_qtd INT)
```

...

Controle de Fluxo - IF

Estrutura de Decisão Condicional

Sintaxe Básica

```
IF   condição   THEN  
    ação_verdadeira;  
  
ELSE  
    ação_falsa;  
  
END IF;
```

Fluxograma Lógico



Exemplo IF

Estrutura de Decisão Lógica



Estrutura de Repetição WHILE

A instrução WHILE executa um bloco de comandos repetidamente enquanto a condição especificada for avaliada como verdadeira.



Avaliação Prévia

A condição é verificada sempre antes da execução dos comandos internos.



Bloco de Comandos

O trecho entre DO e END WHILE forma o ciclo que será repetido.



Critério de Parada

Quando a condição torna-se falsa, o loop é imediatamente encerrado.



script.sql

```
-- Sintaxe básica do loop
```

```
WHILE condição DO
```

```
    comandos;
```

```
END WHILE;
```

Fluxo Lógico

1. Verifica a condição. Se Verdadeiro, continua para o passo 2. Se Falso, sai do loop.
2. Executa os comandos definidos no bloco.
3. Retorna ao passo 1.

EXERCÍCIOS

Práticas de Banco de Dados e Sistemas

01



Procedure info_evento

02



Moradores por bairro

03



Status do evento

04



Total inscrições

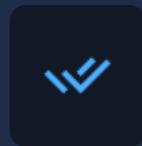
Procedures + Transações

Estruturas fundamentais para controle de fluxo e segurança em bancos de dados



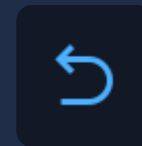
START TRANSACTION

Inicia uma nova transação, isolando as operações subsequentes para garantir que sejam tratadas como uma unidade única.



COMMIT

Efetiva todas as operações realizadas com sucesso, salvando permanentemente as alterações no banco de dados.



ROLLBACK

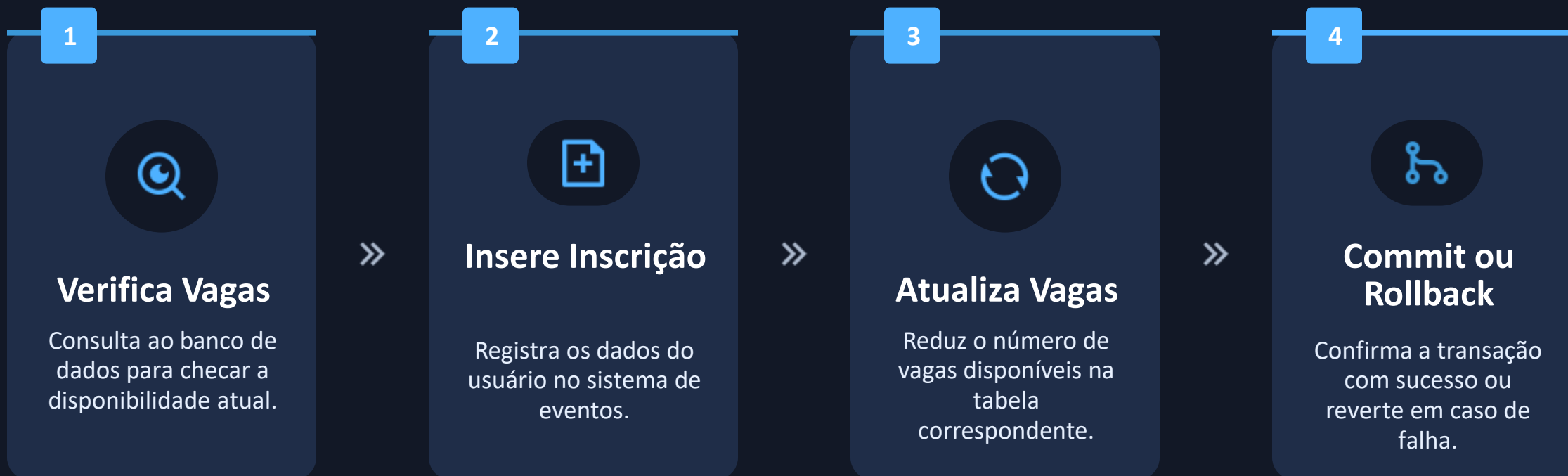
Reverte todas as operações não salvas em caso de falha ou erro, restaurando o estado original antes da transação.



Garantem consistência e integridade dos dados

Atividade Prática

Execute um fluxo contínuo onde a procedure executa as etapas na ordem solicitada.



Conclusão



Quando usar procedures?

Ideais para operações de manipulação de dados em lote, execução de rotinas complexas e redução do tráfego de rede entre a aplicação e o servidor.



Diferença para backend

Procedures executam diretamente no SGBD com altíssima performance local, enquanto o backend é responsável por processar lógicas de negócio na camada da aplicação.



Boas práticas

Mantenha as rotinas bem documentadas, evite regras de negócio excessivas no banco, controle permissões rigorosamente e utilize controle de versão.